Great Expectations Task

1. Install Great Expectations Library

```
!pip install great_expectations
```

Show hidden output

2. Import Necessary Libraries

```
import pandas as pd
import great_expectations as gx
```

Show hidden output

3. Load Labels.csv

Download and upload the Labels.csv into this notebook, and then load the file.

```
# Write code here
from google.colab import files
uploaded = files.upload()
df = pd.read_csv('Labels.csv')
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Labels.csv to Labels.csv
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

4. Preview the Dataset

```
df.head()
```

	Timestamp	Car1_Location_X	Car1_Location_Y	Car1_Location_Z	Car2_Location_X	Car2_Location_Y	Car2_Location_Z	Oc
0	1736796157	-51.402977	143	0.596902	-59.320270	140	0.596902	
1	1736796167	-53.819637	143	0.596902	-59.196568	140	0.596902	
2	1736796178	-50.239144	143	0.596902	-56.744479	140	0.596902	
3	1736796188	-53.707220	143	0.596902	-57.309380	140	0.596902	
4	1736796198	-52.053721	143	0.596902	-59.545897	140	0.596902	

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

5. Set Up Great Expectations Context and Data Source

```
# Write code here

context = gx.get_context()

data_source = context.data_sources.add_pandas(name="labels_datasource")
data_asset = data_source.add_dataframe_asset(name="labels_asset")
```

```
INFO:great_expectations.data_context.types.base:Created temporary directory '/tmp/tmp9hxcqj5s' for ephemeral docs sit
```

6. Define and Create a Data Batch

```
# Write code here

batch_definition = data_asset.add_batch_definition_whole_dataframe(
    name="labels_batch_definition"
)

batch = batch_definition.get_batch(batch_parameters={"dataframe": df})
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

7. Define Three Expectations for Column Values

Using this link, choose three expectation functions and apply them to the labels dataset in a relevant manner.

You should replace the 'ExpectColumnValuesToBeBetween' function with other functions you select from the link.

You can also check the format/parameters required of each function when you click "See more" on the function.

```
## Original Function
expectation = gx.expectations.ExpectColumnValuesToBeBetween(
    column="column", min_value=0, max_value=20
)

## Example Function

## This function only requires a column parameter, and not a max or min value
expectation = gx.expectations.ExpectColumnValuesToBeUnique(
    column="column"
)
```

Expectation 1

```
# Write code here
expectation_1 = gx.expectations.ExpectColumnValuesToBeUnique(
    column="Timestamp"
)
```

Validate Data Against Expectation 1

```
# Write code here
validation_result_1 = batch.validate(expectation_1)
validation_result_1
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

Calculating Metrics: 100% 10/10 [00:00<00:00, 174.18it/s]

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_be_unique",
    "kwargs": {
      "batch_id": "labels_datasource-labels_asset",
      "column": "Timestamp"
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 121,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "partial_unexpected_list": [],
    "missing_count": 0,
    "missing_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "unexpected_percent_nonmissing": 0.0,
    "partial_unexpected_counts": [],
    "partial_unexpected_index_list": []
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

Expectation 2

```
# Write code here
expectation_2 = gx.expectations.ExpectColumnValuesToBeBetween(
    column="pedestrianLocationX_TopLeft",
    min_value=0,
    max_value=1920
)
```

Validate Data Against Expectation 2

```
# Write code here

validation_result_2 = batch.validate(expectation_2)
validation_result_2
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

Calculating Metrics: 100% 10/10 [00:00<00:00, 274.50it/s]

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_be_between",
    "kwargs": {
      "batch_id": "labels_datasource-labels_asset",
      "column": "pedestrianLocationX_TopLeft",
      "min_value": 0.0,
      "max_value": 1920.0
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 121,
```

Expectation 3

```
# Write code here
expectation_3 = gx.expectations.ExpectColumnValuesToNotBeNull(
    column="Ground_Truth_View"
)
```

```
    "unexpected_percent_nonmissing": 0.0,
    "partial_unexpected_counts": [],
    "partial_unexpected_index_list": []
  },
```

Validate Data Against Expectation 3

```
# Write code here
validation_result_3 = batch.validate(expectation_3)
validation_result_3
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

Calculating Metrics: 100% 8/8 [00:00<00:00, 279.79it/s]

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_not_be_null",
    "kwargs": {
      "batch_id": "labels_datasource-labels_asset",
      "column": "Ground_Truth_View"
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 121,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "partial_unexpected_list": [],
    "partial_unexpected_counts": [],
    "partial_unexpected_index_list": []
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```